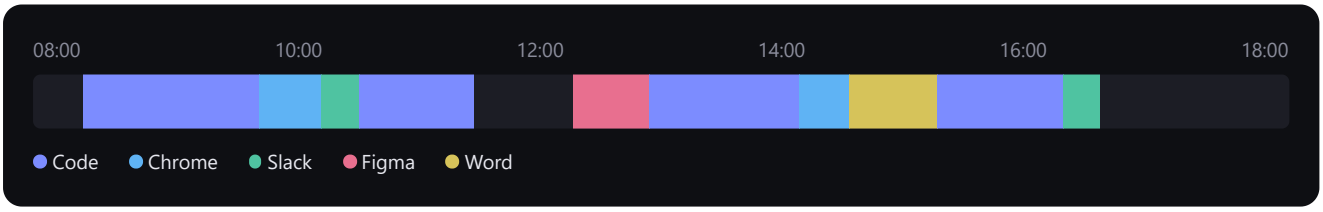
PROJECT GUIDE

Dayframe

See where your day went.

A privacy-first time tracker for Windows. It notices which app you are using, for how long, and turns your day into a timeline you can read at a glance.



MADE BY
Gopinath

VERSION
0.1.0

PLATFORM
Windows 10 / 11

BUILT WITH
Tauri 2 · Rust

© 2026 Gopinath. All rights reserved.

Updated 13 Nov 2026 · github.com/goprocker/Dayframe

CONTENTS

Inside this guide

01	Why Dayframe exists	12	Build, test, debug
02	Every feature	13	Privacy, troubleshooting, uninstall
03	How it looks	14	Dayframe in plain English
04	Tech stack	15	Words you need
05	Architecture	16	Follow the data
06	How tracking works	17	Engineering ideas it shows
07	Data model and time	18	Hard problems and limits
08	Repository map	19	Interview questions and answers
09	Install and run	20	Cheat sheet
10	Using Dayframe	21	Website and wiki
11	Settings	22	CI/CD pipeline

1 s
sampling interval

30 s
max data lost on a crash

1.94 MB
installer size

33
passing tests

> Everything in this guide describes the code in the repository at version 0.1.0. Numbers such as file sizes and test counts were measured on the build this guide was written from.

© 2026 Gopinath. All rights reserved.

SECTION 01

Why Dayframe exists

At six in the evening most people cannot say where the day went. Existing trackers either want an account and a cloud, or read far more than they need: window titles, URLs, even screenshots. Dayframe answers the one question with the least data possible.

Honest numbers

Idle time, the lock screen and sleep are never counted. If you walk away, the clock stops at the moment your hands left the keyboard, not five minutes later.

Private by design

Only the executable and how long it was in front. No titles, no URLs, no keystrokes, no screenshots. No network code at all.

Always on, never in the way

Lives in the tray, starts with Windows if you want, and a tiny optional widget shows the current app. It hides when you go fullscreen.

Yours to delete

One SQLite file on your disk. Exclude apps, delete a date range or everything, withdraw consent any time.

Design goals

Goal	How the code meets it
Correct time	A pure state machine with unit tests for switching, idle, sleep gaps, lock, exclusions, pause and clocks going backwards.
Crash safe	The open session is written to disk every 30 s, so a crash or power cut loses at most 30 s.
Light	One background thread waking once a second; about 32 MB of memory; a 1.94 MB installer.
Consent first	Nothing is recorded until you tick an explicit consent box on first launch.
Local only	A strict Content Security Policy allows only local resources and the app's own IPC.

© 2026 Gopinath. All rights reserved.

SECTION 02

Every feature

Grouped by area. Everything below ships in this repository today.

Tracking

- Foreground app sampled every second
- App name from the executable's own file description (e.g. "Visual Studio Code"), falling back to the file name
- Idle threshold, default 5 min (1 to 120 min); idle time is cut back to when input stopped
- Lock screen, sleep and hibernate never counted
- Pause and resume from the dashboard, widget or tray
- Elevated apps recorded as "Protected application"

Dashboard

- Tiles: active time, apps used, app switches, longest session
- Day timeline with hover details
- App list with real icons, bars, category, sessions and share
- By-hour bars and a clickable 7-day strip
- Previous/next day, date picker, jump to Today
- Today refreshes every 30 s, only while visible

Analytics

- 7, 14 or 30-day ranges
- Total, daily average over active days, top category, top app
- Daily time stacked by category
- Category donut with percentages
- Average by hour of day (DST-correct)
- Top 10 applications

Floating widget

- Transparent, always-on-top pill; off by default
- Status dot, current app, live session timer, pause
- Expands to today's total and top 3 apps
- Drag anywhere; position remembered per monitor layout
- Hides while a fullscreen app is in front

Tray and lifecycle

- Open, Pause/Resume, Show/Hide widget, Settings, Quit
- Closing the window hides it to the tray
- Single instance: launching again focuses the running app
- Start with Windows, straight to the tray
- On quit the open session is closed at quit time

Data and privacy

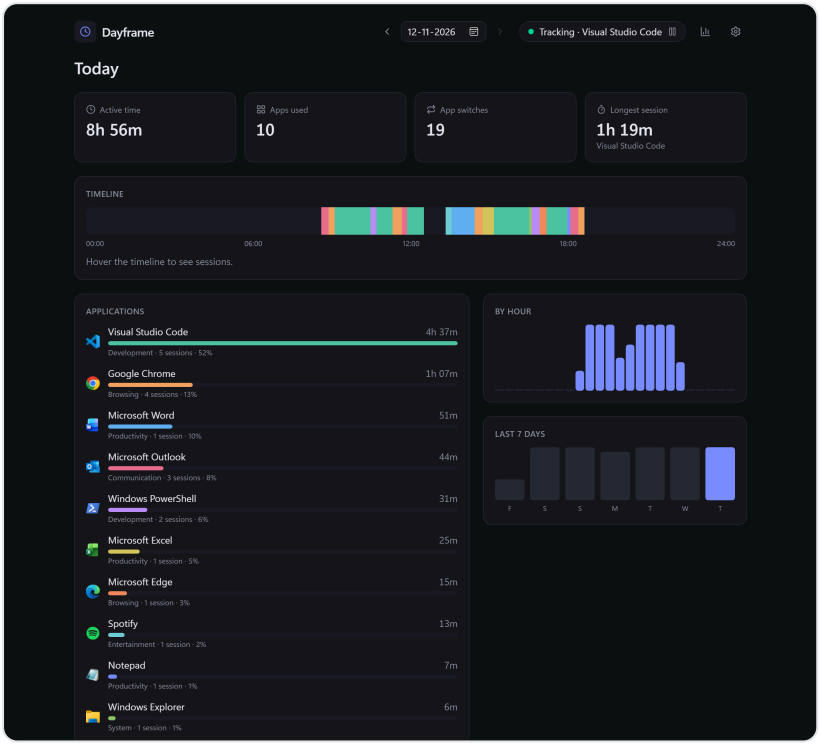
- Explicit consent screen showing the real data folder
- Exclude apps; change an app's category (8 categories)
- Delete a date range or all data, with two-step confirm
- Dark, light or follow Windows

© 2026 Gopinath. All rights reserved.

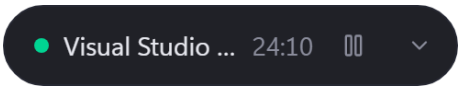
SECTION 03

How it looks

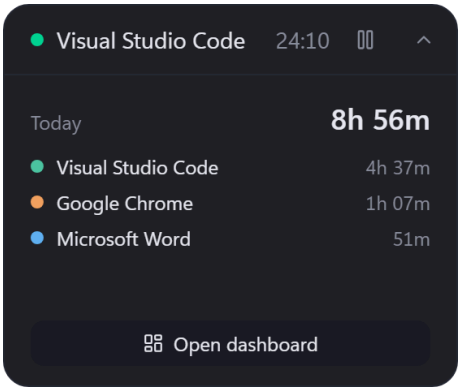
Real screenshots of the app's own screens, rendered with sample data (a typical working day) so no personal history appears in this guide.



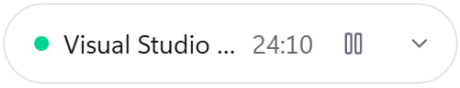
Dashboard (dark): live tracker pill, four tiles, the day timeline, applications with their real icons, by-hour bars and the clickable 7-day strip.



Widget, collapsed (248 × 44): status dot, current app, live session timer, pause, expand.



Widget, expanded (280 × 236): today's total and top three apps. Everything outside the card is transparent.



Widget in the light theme.

Tray menu

Open Dayframe · Pause tracking · Show widget · Settings · Quit Dayframe. The tooltip reads "Dayframe · tracking", "· paused" or "· tracking off". Left-click opens the dashboard.

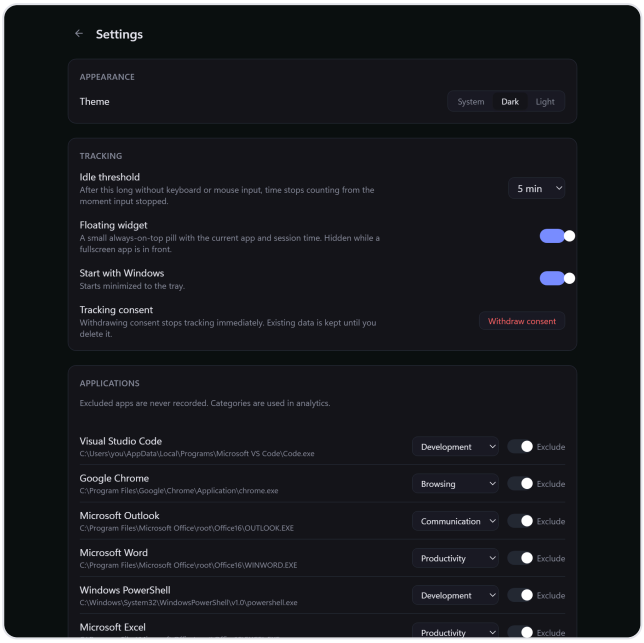
© 2026 Gopinath. All rights reserved.

SECTION 03 · CONTINUED

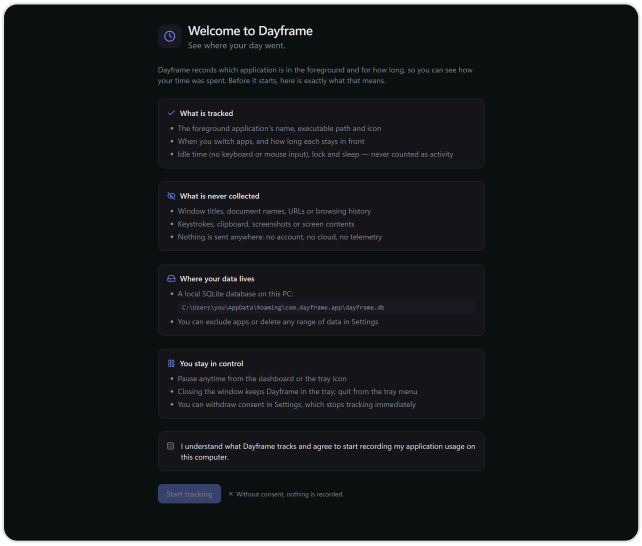
Every screen



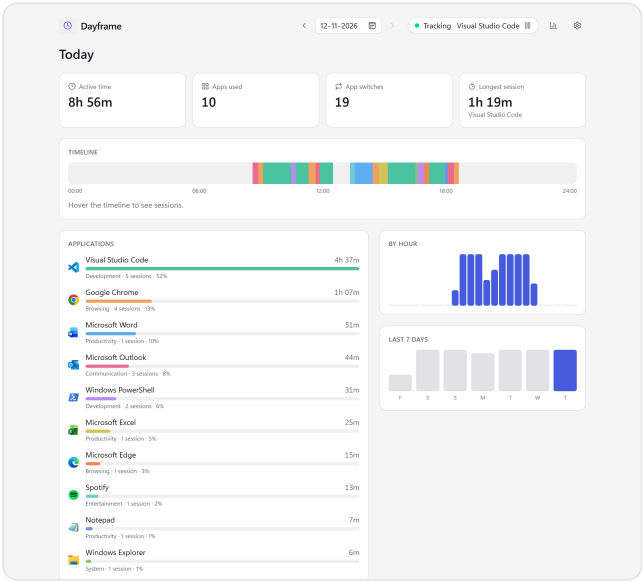
Analytics, 7 days: totals, stacked daily time by category, category donut, average by hour, top apps.



Settings: theme, idle threshold, widget, autostart, consent, and per-app category and exclusion.



First launch: what is tracked, what is never collected, where data lives, and an explicit consent box. The button stays disabled until it is ticked.



The dashboard in the light theme. "System" follows Windows live.

© 2026 Gopinath. All rights reserved.

SECTION 04

Tech stack

A small Rust core does all the work that must be correct and fast; a React frontend draws it. Tauri joins them using the WebView2 already built into Windows.

Layer	Choice	Why
Shell	Tauri 2	Native window, tray, single instance and autostart plugins; uses the system WebView2 so the installer stays under 2 MB (Electron ships a whole browser).
Core	Rust 2021	Memory-safe Win32 calls, a cheap background thread, and fast tests for the state machine.
Win32	`windows` 0.62	Microsoft's official bindings: foreground window, process image name, version resources, idle time, input desktop, monitor info, icons.
Storage	SQLite via `rusqlite` (bundled)	One file, transactional, WAL mode for crash safety; no server.
Time	`chrono`	Local hour of day for analytics across DST changes.
UI	React 19 + TypeScript	Typed command wrappers in `lib/api.ts`; components per screen.
Styling	Tailwind CSS 4	Design tokens as CSS variables; dark and light swap one attribute.
Charts	Recharts 3	Analytics only, lazy-loaded so the dashboard and widget stay small.
Icons	lucide-react	UI icons. App icons come from the executables themselves.
Build	Vite 8, NSIS	Fast dev server on port 1420; per-user installer with no admin rights.
Tests	`cargo test`, Vitest	30 Rust tests and 3 frontend tests.

Sizes measured on 0.1.0

1.94 MB
NSIS installer

4.6 MB
`dayframe.exe` (release, LTO, stripped)

~32 MB
memory while tracking

74 KB gz
main JS bundle

112 KB gz
Analytics chunk (loaded on demand)

1.8 KB gz
Widget chunk

© 2026 Gopinath. All rights reserved.

SECTION 05

Architecture

One process. A tracker thread samples Windows, a pure engine decides what happened, and SQLite remembers it. The windows only read.



Shared state

Thing	Type	Who touches it
Database	`Mutex<Connection>`	Tracker thread writes; commands read and write; icon extraction happens outside the lock.
Control	atomics	`consented`, `paused`, `reload`, `reopen`, `stop`: flipped by commands and tray, read by the loop each tick.
Snapshot	`Mutex<Snapshot>`	Status, app name, session start. Read by `tracker_state`.
WidgetState	atomics	Enabled flag and current fullscreen state.

Startup order matters: open DB → recover → read consent and widget flag → manage state → spawn tracker → build tray → show widget → show main window unless launched with `--minimized`. On quit, `stop` is set and the thread is joined so the open session ends at quit time.

© 2026 Gopinath. All rights reserved.

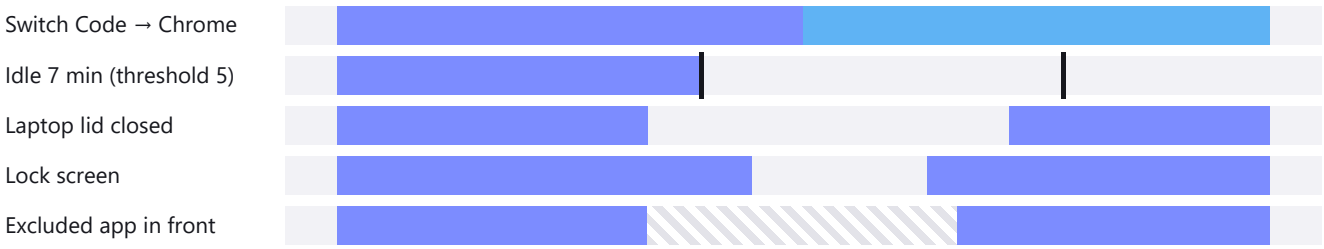
SECTION 06

How tracking works

Each second the loop hands the engine one `Sample { now_ms, app, idle_ms, locked }`. The engine answers with zero or more actions. These rules are the whole product.

Situation	What the engine does
Same app, still active	Nothing, except a Checkpoint every 30 s that moves the session end forward.
App switch	Close the old session at `now` and open the new one at `now`. Sessions never overlap.
Idle ≥ threshold	Close at `now − idle_ms`: the moment input stopped. Idle time is never counted.
Idle below threshold	Still counted. Reading or watching without touching the mouse is real use.
Gap > 15 s between ticks	The PC slept or hibernated. Close at the last tick, not at wake-up.
Clock went backwards	Treated like a gap; never produces a negative session.
Locked, paused, excluded, no app	Close at `now`; reopen when the condition ends.

Five scenarios on a timeline



Coloured = counted. Line 2: the session ends where input stopped (first mark), not where idle was detected (second mark). Hatched = not recorded.

Why a pure engine?

Every rule above is a function of the sample and the engine's memory, so it can be tested with fake timestamps in microseconds instead of waiting minutes for a real idle. The Win32 code and the database stay outside; the loop just glues them. Nine engine tests cover exactly the rows in the table.

© 2026 Gopinath. All rights reserved.

SECTION 07

Data model and time

Four tables, UTC milliseconds everywhere, and day boundaries decided by the screen that knows your calendar.

```
applications       id · exe_key UNIQUE · display_name · executable_path · category
                   excluded · icon_png (cached) · first_seen_at
activity_sessions  id · application_id → applications (CASCADE) · started_at · ended_at
                   duration_seconds · project_id → projects · CHECK (ended_at >= started_at)
projects           id · name · color · created_at          (reserved for later)
settings           key · value                            (consent, theme, idle, widget …)
indexes            started_at · ended_at · application_id
```

Always stored closed

A session row is inserted with `ended_at = started_at` and pushed forward by each checkpoint. After a crash the row is already valid; startup recovery only recomputes `duration_seconds`.

Migrations

An append-only `MIGRATIONS` list tracked by `PRAGMA user_version`. Never edit an applied one; add the next.

Clipping

`intervals(from, to)` returns every session overlapping the window, cut to it. A session from 23:30 to 00:45 gives 30 min to one day and 45 to the next.

Deleting a range

`delete_range` trims or splits sessions that straddle the edges, so only the chosen time disappears.

Local days and DST

The database never decides what "today" is. The frontend computes local midnights (`lib/time.ts`) and passes them as bounds. On the day clocks go forward a day is 23 hours, so the hourly chart gets 23 buckets; on the day they go back, 25. The analytics "average by hour" maps each bucket to its real local hour with `chrono::Local`, so 02:00 on a DST day is not mislabelled.

Deletion never comes back

If you delete data while a session is open, the next checkpoint could extend a row that no longer exists, or re-create time you just removed. The delete command sets `reopen` while still holding the DB lock; the tracker checks it under the same lock and forgets its open session before writing anything else.

© 2026 Gopinath. All rights reserved.

SECTION 08

Repository map

Where everything lives. Rust in `src-tauri/src`, React in `src`.

Path	What it does
src-tauri/src/lib.rs	App setup, managed state, commands for stats and icons, startup and shutdown.
src-tauri/src/tracker/engine.rs	The pure tracking state machine and its tests.
src-tauri/src/tracker/win.rs	Win32 sampling: foreground app, idle, lock, fullscreen.
src-tauri/src/tracker/mod.rs	The 1 s loop, Control flags, Snapshot, DB writer.
src-tauri/src/db.rs	Schema, migrations, queries, clipping, deletion, recovery.
src-tauri/src/stats.rs	Day summary, timeline merge, hourly buckets, range analytics.
src-tauri/src/icons.rs	Executable icon → PNG, cached in the DB.
src-tauri/src/categories.rs	Default category from the exe name; the 8 categories.
src-tauri/src/settings.rs	Settings, consent, exclusions, categories, deletion commands.
src-tauri/src/tray.rs	Tray menu, labels and tooltip.
src-tauri/src/widget.rs	Widget window, position, fullscreen hiding.
src-tauri/tauri.conf.json	Window, CSP, bundle (NSIS) config.
src-tauri/capabilities/	Which windows may call what.
src/main.tsx	Picks Widget or App by window label.
src/App.tsx	Consent gate, view switch, theme.
src/components/	Dashboard, Timeline, AppIcon, Analytics, Settings, Onboarding, Widget.
src/lib/api.ts	Typed wrappers for every command and event.
src/lib/time.ts	Local-day bounds and formatting (with Vitest tests).
src/lib/colors.ts	Per-app and per-category colours.
src/index.css	Tailwind and the design tokens for both themes.
README.md · ARCHITECTURE.md	Overview and design notes.
SESSION_HANDOFF.md	Build history and what was verified, chunk by chunk.

© 2026 Gopinath. All rights reserved.

SECTION 09

Install and run

Use the installer, or build from source in a few commands.

Installer

1. Run `Dayframe_0.1.0_x64-setup.exe`. It installs for your user only; no admin rights.
2. If WebView2 is missing (rare on Windows 10/11) the installer downloads it.
3. Dayframe opens on the consent screen. Nothing is tracked until you agree.

From source

You need Windows 10/11, Node.js 20+, Rust stable with the MSVC toolchain and the Tauri prerequisites (Visual Studio Build Tools with "Desktop development with C++").

```
git clone https://github.com/goprocker/Dayframe
cd Dayframe
npm install
npm run tauri dev        # run with hot reload (Vite on port 1420)
npm run tauri build      # release exe + installer
```

The installer lands in `src-tauri/target/release/bundle/nsis/`. The first Rust build takes several minutes; later ones are incremental.

Where things are on disk

What	Location
Database	`%APPDATA%\com.dayframe.app\dayframe.db` (plus `-wal` and `-shm` files)
WebView data	`%LOCALAPPDATA%\com.dayframe.app\`
Autostart	A Run registry entry for your user, launching with `--minimized`

© 2026 Gopinath. All rights reserved.

SECTION 10

Using Dayframe

A normal day, from first launch to reading the week.

First launch

1. The onboarding screen lists what is tracked, what is never collected, and the real folder where data is stored.
2. Tick the consent box and press the button to start. Tracking begins on the next second.
3. You land on the dashboard. It fills in as you use other apps.

Everyday

Glance

The pill in the header says what is being tracked now: Tracking · app, Idle, Paused, Locked, Excluded app, or Waiting.

Pause

Use the pause button in the header pill, the widget, or the tray. The open session closes immediately; nothing is counted until you resume.

Close the window

It hides to the tray and keeps tracking. Use tray › Quit to stop completely.

Look back

Arrows, the date picker, or a click on any bar in the 7-day strip. "Today" jumps back.

Timeline

Hover a block for the app, start, end and duration. Same-app pieces less than 2 s apart are merged.

Analytics

The chart button in the header. Choose 7, 14 or 30 days.

The widget

1. Turn it on from tray › Show widget, or Settings › Floating widget.
2. It appears at the top-right of your main screen. Drag it by any empty part.
3. The chevron expands it to today's total and your top three apps; "Open dashboard" brings the main window forward.
4. Start a fullscreen video or game and it disappears; it returns when you leave fullscreen.

Reading the numbers

App switches counts changes between timeline blocks. **Longest session** is the longest merged block of one app. **Daily average** in Analytics divides by days that had any activity, so a weekend off doesn't drag it down.

© 2026 Gopinath. All rights reserved.

SECTION 11

Settings

Open from the gear in the dashboard header or tray › Settings. Changes apply immediately.

Setting	Options	Effect
Theme	System · Dark · Light	System follows Windows live. Both windows switch together.
Idle threshold	1, 2, 3, 5, 10, 15, 30 min	How long without input before time stops counting. The tracker reloads it on the next tick.
Floating widget	On · Off	Shows or hides the always-on-top pill. Same as the tray item.
Start with Windows	On · Off	Launches to the tray at sign-in.
Tracking consent	Withdraw	Stops tracking at once and returns to onboarding. Existing data is kept.
Applications	Exclude · Category	Excluded apps are never recorded from now on. Categories feed Analytics.
Delete a range	From and to dates	Removes tracked time in those days, trimming sessions at the edges.
Delete all data	Confirm twice	Removes every session and app record, then compacts the file. Settings stay.
Privacy	—	A plain summary of what is and isn't recorded, next to the data folder.

Categories

Development · Code, Visual Studio, JetBrains IDEs, terminals, Docker Desktop, Postman …

Browsing · Chrome, Edge, Firefox, Brave …

Communication · Slack, Teams, Discord, Zoom, Outlook, WhatsApp …

Design · Figma, Photoshop, Illustrator, Blender …

Productivity · Word, Excel, PowerPoint, Notion, Obsidian, Acrobat …

Entertainment · Spotify, VLC, Steam, Epic Games …

System · Explorer, Settings, Task Manager, Control Panel

Other · anything not recognised

Defaults are guessed from the executable name in `categories.rs`; every app can be changed in Settings.

Confirmations without pop-ups

Destructive buttons switch to a "Confirm?" state inline instead of opening a native dialog. Native dialogs block the WebView and are easy to click through by habit.

© 2026 Gopinath. All rights reserved.

SECTION 12

Build, test, debug

Three commands cover nearly everything.

```
cd src-tauri && cargo test   # 30 Rust tests
npx vitest run               # 3 frontend tests
npm run build                # TypeScript check + Vite build
```

What the tests cover

Area	Tests
Engine (9)	Switching without overlap, idle cut-back, below-threshold idle, sleep gap, lock/unlock, exclusions, pause/resume, periodic checkpoints, clock going backwards.
Database	Idempotent migrations, app upsert, interval clipping, `end ≥ start`, crash recovery, range deletion, settings.
Stats	Totals, switches, longest block, hourly split, 23/25-hour DST days, midnight-spanning session, empty day, range merge, local-hour mapping.
Windows	Exe-name fallback, reading Notepad's real file description and icon, idle ≥ 0.
Widget	Parsing the saved position.
Frontend	Local-day bounds and duration formatting.

Debugging tips

WebView2 0x800700AA

Killing a dev run can leave `msedgewebview2.exe` holding the data folder, so the next launch panics. End those processes, then relaunch.

Inspect the data

Open `dayframe.db` with any SQLite browser while the app is closed. Times are UTC ms.

DevTools

Right-click › Inspect in a dev build to see console errors and command calls.

Logs

DB write failures in the tracker are logged with `log::error!` and never crash the loop.

Commit conventions

Conventional Commit subjects with a body derived from the actual diff; one logical change per commit; work is shipped in chunks, each ending with an updated `SESSION_HANDOFF.md`.

© 2026 Gopinath. All rights reserved.

SECTION 13

Privacy, troubleshooting, uninstall

Collected

- Executable path and its file description
- Start and end time of each session
- Whether you were idle, locked or paused (to stop counting)
- The app's icon, cached for display

Never collected

- Window titles, document or tab names
- URLs or browsing history
- Keystrokes or mouse positions
- Screenshots or screen contents

There is no network code. The Content Security Policy is `default-src 'self'` with only local images and IPC allowed, so even a bug in the UI could not send data out.

Troubleshooting

Symptom	Fix
Nothing is being recorded	Check the header pill. "Paused" → resume. "Excluded app" → un-exclude in Settings. Onboarding showing → consent was withdrawn.
Less time than expected	Idle threshold may be short; raise it. Time away from the keyboard past the threshold is removed on purpose.
An app shows as "Protected application"	It runs elevated (as administrator). Windows does not let a normal process read its path.
Widget not visible	Turn it on from the tray. If a fullscreen window is in front it hides until you leave fullscreen.
Widget off-screen after unplugging a monitor	The saved position is only reused if it lies on a connected monitor; otherwise it returns to the top-right.
Second window opens instead of the first	It shouldn't: single-instance focuses the running app. Check for an old version in Task Manager.

Uninstall

1. Quit from the tray.
2. Settings › Apps › Installed apps › Dayframe › Uninstall. Tick "Delete the application data" to remove the database too.
3. Or delete `%APPDATA%\com.dayframe.app` and `%LOCALAPPDATA%\com.dayframe.app` yourself.

© 2026 Gopinath. All rights reserved.

SECTION 14

Dayframe in plain English

No code. If you can explain these paragraphs, you understand the app.

> Imagine someone sitting beside you with a stopwatch and a notebook. Every second they glance at your screen and note only the name of the program in front. When you switch programs they write down when the last one stopped and the new one started. If you stop touching the keyboard for a while, they cross out the time back to when you stopped. If you lock the PC or close the lid, they stop writing.

That is Dayframe. The "glance" is a Windows call that names the program in front. The "notebook" is a small database file in your user folder. Every 30 seconds the notebook is saved, so even if the power goes out you lose at most half a minute.

The dashboard is the notebook turned into pictures: a coloured strip for the day, bars for each app, and how busy each hour was. Analytics does the same over weeks.

The widget is a sticky note on your screen showing what the stopwatch is timing right now. The tray icon near the clock is how you reach the app when its window is closed; closing the window does not stop the stopwatch.

Three promises

It only looks at names

Never what is inside a window: no titles, pages or keys.

It never counts away time

Idle, locked or asleep is not work, so it is not recorded.

It never leaves your PC

There is nothing to upload to. Delete the file and it's gone.

© 2026 Gopinath. All rights reserved.

SECTION 15

Words you need

Word	Meaning here
Foreground window	The window receiving your keyboard input; the "app in front".
Executable / exe	The program file, e.g. `Code.exe`. Dayframe identifies apps by its path (`exe_key`).
File description	A friendly name stored inside an exe's version resource ("Visual Studio Code").
Session	One continuous stretch of one app, stored as a start and end time.
Interval	A session cut to a time window, e.g. one day.
Block	Neighbouring intervals of the same app merged for the timeline.
Checkpoint	Moving the open session's end forward on disk every 30 s.
Idle threshold	How long without input before time stops counting.
State machine	Code that keeps a little memory and reacts to inputs by rules; the engine.
Pure function	Same inputs, same outputs, no side effects. Easy to test.
Tauri	A framework for desktop apps with a Rust backend and a web UI.
WebView2	Microsoft Edge's engine embedded in apps; draws Dayframe's UI.
Command / invoke	A Rust function the UI can call, like a local API.
Event / emit	A message Rust pushes to the UI, e.g. `tracker://state`.
Capability	Tauri's allow-list of what each window may call.
CSP	Content Security Policy: rules for what the UI may load or connect to.
SQLite / WAL	An embedded database in one file; Write-Ahead Logging makes writes crash-safe.
Migration	A numbered schema change applied once, in order.
Mutex / atomic	Ways for threads to share data safely: a lock, or a single value changed in one step.
UTC ms	Milliseconds since 1970 in UTC; the same everywhere, no time zones.
DST	Daylight saving time; makes some local days 23 or 25 hours long.
NSIS	The tool that builds the Windows installer.
Tray	The icon area by the clock on the taskbar.

© 2026 Gopinath. All rights reserved.

SECTION 16

Follow the data

One second in the life of Dayframe: you click into VS Code, and 30 seconds later it shows on the dashboard.

1. **Sample.** The tracker thread wakes. `GetForegroundWindow` → `GetWindowThreadProcessId` → `OpenProcess` (limited query rights) → `QueryFullProcessImageNameW` gives `C:\…\Code.exe`. The description "Visual Studio Code" comes from a cache, read once from the version resource.
2. **Idle and lock.** `GetLastInputInfo` vs `GetTickCount` gives idle ms (wrap-safe). `OpenInputDesktop` named "Default" means unlocked.
3. **Decide.** `engine.tick(sample)` sees a different app than before and returns `[Close(now), Open(Code, now)]`.
4. **Write.** The loop locks the DB, upserts the app (category guessed as Development), ends the old row at `now` and inserts a new one starting at `now`.
5. **Tell.** The new Snapshot (Active, "Visual Studio Code", start) differs from the last one, so `tracker://state` is emitted. The header pill and widget update; the widget timer starts ticking locally.
6. **Keep.** Every 30 s a Checkpoint moves `ended_at` forward.
7. **Show.** The dashboard's 30 s refresh calls `day_summary(from, to)` with local midnights. `db::intervals` clips sessions to the day; `stats::summarize` builds totals, per-app usage, hourly buckets and merged timeline blocks.
8. **Icon.** The app list sees an app without an icon and calls `app_icons`. `ExtractIconExW` → bitmap → PNG, done outside the DB lock, then cached forever.

And when you walk away

```
t = 10:00:00   last input
t = 10:05:00   idle_ms = 300000 ≥ threshold → Close(end = 10:00:00)
               status Idle; snapshot emitted; widget dot turns grey
t = 10:12:40   input again, same app → Open(Code, 10:12:40)
```

The 12 minutes 40 seconds away are never written, and the 5 minutes it took to notice are taken back.

© 2026 Gopinath. All rights reserved.

SECTION 17

Engineering ideas it shows

Functional core, imperative shell

The engine and stats are pure; Win32, SQLite and Tauri sit at the edges. Most bugs are caught by fast tests on the core.

Store facts, derive views

Only sessions are stored. Totals, switches, hourly charts and analytics are computed on read, so a change in how they're shown never needs a migration.

Durable by default

Rows are always closed and checkpointed; recovery is a one-line normalisation, not a repair job.

Bounded loss

A clear, testable promise: a crash loses at most 30 s. Cheaper than writing every second, safer than writing on quit.

Ordering under a lock

Deletion and the tracker's "forget" happen under the same DB lock, closing a race that would resurrect deleted time.

Push, don't poll

The UI gets `tracker://state` only when something changes; the dashboard refreshes only while visible; the widget fetches totals only while expanded.

Calendar at the edge

UTC in storage; local days computed by the UI and passed in. DST becomes a bucket count, not a bug.

Pay for what you use

Recharts is lazy-loaded; the widget window is created only when enabled; icons are extracted once.

Least privilege

Limited process-query rights, no titles, a strict CSP, per-window capabilities, consent before the first sample.

Append-only migrations

Versioned with `user_version`; old databases upgrade in order, tested to be idempotent.

© 2026 Gopinath. All rights reserved.

SECTION 18

Hard problems and limits

Problem	What Dayframe does	Limit
Detecting sleep	A gap of more than 15 s between 1 s ticks means the machine was suspended; close at the last tick.	Not using power broadcast messages; a 15 s system freeze also counts as a gap.
Detecting lock	If the input desktop isn't "Default" (or can't be opened), the session is locked.	Not using WTS session notifications; UAC prompts also count as locked, briefly.
Idle while reading	Below the threshold, no input still counts as use.	A long video without touching anything ends at the threshold; raise it if needed.
Elevated apps	Recorded as one "Protected application".	Can't tell elevated apps apart without running elevated, which Dayframe won't.
Midnight	Sessions are clipped to each day's bounds.	—
DST	23/25 buckets; local hour mapping in analytics.	The 7-day strip shows totals only, unaffected.
Fullscreen	Foreground rect covers its monitor; desktop and shell windows excluded.	A borderless maximised window that exactly covers the screen also hides the widget.
Window creation	The widget is created from setup, the tray or an async command.	Creating a window from a synchronous command deadlocks WebView2 on Windows.
Multiple monitors	The widget's saved position is reused only if it lies on a connected monitor.	No per-monitor memory yet.
Browsers	Counted as one app.	By design: no tab titles or URLs, so no per-site breakdown.

Not built yet

- Projects: the table exists; there is no UI to assign time to projects.
- Export to CSV.
- Goals and daily limits.
- macOS and Linux.

© 2026 Gopinath. All rights reserved.

SECTION 19

Interview questions and answers

Why Tauri and not Electron?

Tauri uses the WebView2 already in Windows, so the installer is under 2 MB and idle memory is about 32 MB. The backend is Rust, which suits Win32 calls and a long-running background thread.

How do you avoid counting idle time?

When idle reaches the threshold, the engine closes the session at `now - idle_ms`: the moment input stopped. So the time spent waiting to notice is removed as well.

What happens if the app crashes?

Sessions are stored closed and checkpointed every 30 s. On startup recovery only recomputes durations. Worst case: 30 s lost.

How do you detect sleep without power events?

The loop ticks every second. If more than 15 s passed, or the clock went backwards, the machine was suspended; the session ends at the last tick.

How do you handle sessions across midnight and DST?

Store UTC ms. The UI computes local midnights and passes bounds; sessions are clipped to them. Bucket count equals the day's real length; analytics map buckets to local hours with chrono.

How did you make the tracker testable?

A pure state machine takes a sample struct and returns actions. Tests feed fake timestamps; Win32 and SQLite are outside it.

Describe a concurrency bug you prevented.

Deleting data while a session is open could let the next checkpoint extend a deleted row. The delete sets a flag under the DB lock; the tracker checks it under the same lock and forgets its session first.

Why not store totals per day?

Derived data goes stale when you delete or re-categorise. Sessions are the single source of truth; aggregation is fast enough on read with indexes on start and end.

How is privacy enforced, not just promised?

The sampler never calls title APIs; there's no network code; the CSP blocks outbound connections; tracking is gated on stored consent.

How does the widget hide in fullscreen?

While enabled, each tick compares the foreground window's rect with its monitor's rect (ignoring the desktop and shell). Changes toggle the window's visibility.

What would you do next?

WTS and power notifications instead of heuristics, projects, CSV export, and per-monitor widget positions.

© 2026 Gopinath. All rights reserved.

SECTION 20

Cheat sheet

Commands

```
npm install
npm run tauri dev
npm run tauri build
npm run build
npx vitest run
cd src-tauri && cargo test
```

Numbers

Tick	1 s
Checkpoint	30 s
Sleep gap	> 15 s
Idle default	5 min (1–120)
Timeline merge	gap ≤ 2 s
Dashboard refresh	30 s, visible only
Widget	248×44 / 280×236
Analytics range	≤ 92 days
Day range	≤ 26 h

Tauri commands

`app_info` · `tracker_state` · `day_summary` · `range_totals` · `range_summary` · `app_icons` · `get_settings` · `set_tracking_consent` · `set_tracking_paused` · `set_theme` · `set_idle_threshold` · `set_autostart` · `list_apps` · `set_app_excluded` · `set_app_category` · `delete_range` · `delete_all_data` · `set_widget_enabled` · `set_widget_expanded` · `save_widget_position` · `open_dashboard`

Events

`tracker://state` (snapshot on change) · `navigate` (tray → Settings) · `theme` (keeps windows in sync)

Settings keys

`tracking_consented` · `theme` · `idle_threshold_ms` · `widget_enabled` · `widget_position`

Practice

1. Add a test: idle that ends exactly at the threshold.
2. Add a "Copy as CSV" button for a day.
3. Replace the sleep heuristic with `WM_POWERBROADCAST`.
4. Build a projects picker on the timeline.

© 2026 Gopinath. All rights reserved.

SECTION 21

Website and wiki

Dayframe has a public landing page at **dayframe.loopfound.in** and a wiki on GitHub. Both live next to the app but are built separately.

The landing page

Plain HTML, CSS and JavaScript in `landing/`, built with Vite and hosted on Vercel. It has its own version (0.9.0) and changelog, separate from the app.

What's on it

An intro loader that fills a 24-hour timeline, a painted meadow hero with a retro TV playing the product video, a scroll-driven tour, the "A day, framed" story, screenshots and the installer download.

Changelog page

A second page at `/changelog` lists every landing release. Vite builds it as a second entry next to `index.html`.

Downloads

The installer is served from `/download/` as a file download. Clicks add one to an anonymous public counter. Localhost and the owner's browsers (`?me=1`) are not counted.

Deploy

The Vercel project's root directory is `landing`. `vercel.json` sets the build, clean URLs and download headers.

Commands

```
cd landing
npm install
npm run dev      # http://localhost:5180
npm run build    # landing/dist
```

The wiki

github.com/goprocker/Dayframe/wiki has short pages for Installation, Using Dayframe, Privacy, Building from Source, Architecture, Landing Page and Troubleshooting. Use it as the quick reference; this guide is the long version.

© 2026 Gopinath. All rights reserved.

SECTION 22

CI/CD pipeline

Every change is built and tested automatically on GitHub, and a version tag turns into a downloadable Windows installer. Workflows live in `.github/workflows/`.

The idea

CI (continuous integration) builds and tests each push and pull request so a broken change is caught before it is merged. **CD** (continuous delivery) packages a passing build into something you can ship. Dayframe uses GitHub Actions for both.

ci.yml: runs on every pull request and push to main

Job	Runs on	Steps
frontend	Ubuntu	`npm ci`, `npm run build` (TypeScript check + Vite build), `npm test` (vitest)
landing	Ubuntu	`npm ci` and `npm run build` inside `landing/`
rust	Windows	builds the frontend (Tauri needs `dist/`), then `cargo fmt --check`, `cargo clippy`, `cargo test`

The Rust job runs on Windows because the tracker uses Win32 APIs. npm and cargo caches speed up repeat runs, and a newer push to the same branch cancels the older run. Clippy currently reports warnings without failing the job; it becomes strict once the two existing lints in `icons.rs` are fixed.

release.yml: runs when you push a version tag

1. Bump the version in `package.json`, `src-tauri/Cargo.toml` and `src-tauri/tauri.conf.json`, then merge.
2. Tag and push: `git tag v1.0.1 && git push origin v1.0.1`.
3. The workflow builds the NSIS installer with `tauri-action` on a Windows runner.
4. It attaches the installer to a **draft** GitHub Release. Review the notes, then publish.

The workflow can also be started by hand from the Actions tab. The installer is not code-signed, so Windows SmartScreen may warn on first run.

Dependabot

`.github/dependabot.yml` opens weekly update pull requests for the app's npm packages, the landing site's npm packages and the Rust crates in `src-tauri`. Tauri packages are grouped into one pull request. CI runs on each, so an update only merges when the build and tests pass.

Before you open a PR

Run `npm run build`, `npm test`, `cargo fmt` and `cargo test` locally. They are exactly what CI runs.

If CI fails

Open the failing job in the pull request's Checks tab, read the first red step, and reproduce it locally with the same command.

© 2026 Gopinath. All rights reserved.